

TRƯỜNG ĐẠI HỌC FPT

Viện Đào tạo Sau Đại học

**Kiến trúc Nhận thức Hai tầng cho
Phát hiện và Phản hồi Mối đe dọa
Tự động sử dụng AI Tác tử**

Đề cương Luận văn Thạc sĩ

Học viên: Nguyễn Đức Bình

MSSV: MSE13183

Giảng viên hướng dẫn: Bùi Văn Hiệu

Hà Nội, 2026

Tóm tắt

Trong bối cảnh an ninh mạng hiện đại, các Trung tâm Điều hành An ninh (SOC) đang phải đối mặt với một “Nghịch lý Dữ liệu lớn” nghiêm trọng: khối lượng khổng lồ các sự kiện mạng phát sinh mỗi giây dẫn đến tình trạng quá tải cảnh báo, với phần lớn trong số đó là dương tính giả. Các hệ thống Phát hiện Xâm nhập (IDS) và Quản lý Sự kiện An ninh (SIEM) truyền thống thường phân tích luồng dữ liệu một cách đơn lẻ, qua đó làm suy giảm đáng kể khả năng nhận diện các chiến dịch tấn công có chủ đích tinh vi (APT). Gần đây, việc ứng dụng trực tiếp các Mô hình Ngôn ngữ Lớn (LLM) vào việc phân tích log đã được thử nghiệm; tuy nhiên, hướng đi này lại mở ra hai lỗ hổng rủi ro lớn: độ trễ suy luận quá cao không thể đáp ứng được tốc độ của luồng dữ liệu thời gian thực, và nguy cơ hệ thống LLM bị thao túng bởi các đòn tấn công Prompt Injection ẩn giấu ngay bên trong dữ liệu log độc hại.

Để giải quyết triệt để những thách thức này, nghiên cứu đề xuất một kiến trúc nhận thức (cognitive architecture) kết hợp giữa một Động cơ Quy tắc (Rule Engine) xác định và một Mô hình Ngôn ngữ Lớn. Thông qua phương pháp tiếp cận hai tầng, hệ thống được kỳ vọng sẽ sàng lọc lượng lớn nhiễu ở tốc độ cao (Tầng 1), đồng thời bảo lưu năng lực suy luận sâu — được vận hành bởi AI Tác tử dựa trên LangGraph cùng cơ chế Dual-RAG (tích hợp MITRE ATT&CK cho nhận diện mối đe dọa và NIST SP 800-61r2 cho hướng dẫn ứng phó sự cố) — để xử lý các mối đe dọa phức tạp và khó đoán (Tầng 2). Bên cạnh đó, các cơ chế an toàn nâng cao được triển khai bao gồm: Đóng gói Dữ liệu có Phân cách (Delimited Data Encapsulation) chống Prompt Injection, xác thực mã ký SHA-256 bảo vệ tính toàn vẹn của cơ sở tri thức tĩnh chống RAG Ingestion Poisoning, làm sạch cấu trúc ngữ cảnh để ngăn chặn đầu độc bộ nhớ đệm (Semantic Cache Poisoning), bảo mật biểu mẫu xác thực bằng Regex và thuật toán PBKDF2-HMAC-SHA256, cùng cơ chế ký HMAC chuỗi logs bảo vệ tính toàn vẹn của nhật ký kiểm toán.

Hệ thống đề xuất sẽ được triển khai và đánh giá trên hai bộ dữ liệu chuẩn: CSE-CIC-IDS2018 cho phát hiện xâm nhập mạng doanh nghiệp và DAPT2020 cho theo dõi tấn công APT đa giai đoạn. Các tiêu chí đánh giá sẽ được cấu trúc theo Khung Đánh giá 5 Chiều (5-Dimensional Evaluation Framework) toàn diện, bao quát Chỉ số Phân loại, Chỉ số Vận hành, Mức độ Kháng lỗi, Chất lượng Ngữ cảnh và Tính Giải thích.

Mục lục

Tóm tắt	1
1 Giới thiệu và Bối cảnh	4
1.1 Mô tả Vấn đề	4
1.2 Mục tiêu Nghiên cứu	4
1.3 Câu hỏi Nghiên cứu	5
1.4 Đối tượng và Phương pháp Nghiên cứu	5
1.5 Phạm vi Nghiên cứu	5
1.6 Cấu trúc Luận văn	6
2 Tổng quan Nghiên cứu	7
2.1 Các Phương pháp Nền tảng	7
2.2 Các Công trình Liên quan	7
2.2.1 Phân tích So sánh các Phương pháp Hiện có	8
3 Giải pháp Đề xuất và Phương pháp Nghiên cứu	9
3.1 Giải pháp Đề xuất: Kiến trúc Nhận thức Hai tầng	9
3.2 Tài liệu Kiến trúc Kỹ thuật Toàn diện	11
3.2.1 PHẦN 1: DATASET LAYER	11
3.2.2 PHẦN 2: TIER 1 — RULE ENGINE	14
3.2.3 PHẦN 3: GUARDRAIL LAYER	16
3.2.4 PHẦN 4: TIER 2 — LANGGRAPH AGENT	19
3.2.5 PHẦN 5: HITL DASHBOARD	24
3.2.6 PHẦN 6: EVALUATION FRAMEWORK (5D)	27
3.3 Công nghệ và Công cụ	30
3.3.1 Cấu hình Phần cứng	30
4 Kế hoạch Triển khai và Đánh giá	31

4.1	Các Giai đoạn Phát triển Chính	31
4.2	Kế hoạch Đánh giá	31
4.3	Kế hoạch Thực hiện	32
5	Kết quả Kỳ vọng và Đóng góp Khoa học	34
5.1	Kết quả Kỳ vọng và Thực nghiệm	34
5.2	Đóng góp Khoa học	34
6	Kết luận	36
	Tài liệu tham khảo	37

1. Giới thiệu và Bối cảnh

1.1 Mô tả Vấn đề

Các Trung tâm Điều hành An ninh (SOC) hiện đại đang bị nhấn chìm bởi khối lượng sự kiện an ninh khổng lồ. Các báo cáo chuyên ngành gần đây chỉ ra rằng một phân tích viên SOC trung bình phải xử lý hàng nghìn cảnh báo mỗi ngày, trong đó gần một nửa bị xác định là dương tính giả [1]. Hiện tượng “quá tải cảnh báo” (alert fatigue) này trực tiếp dẫn đến việc các mối đe dọa nguy hiểm thực sự bị bỏ sót. Thêm vào đó, các hệ thống IDS/SIEM truyền thống thường phân tích từng nguồn dữ liệu (Web log, Firewall log) một cách rời rạc, làm giảm nghiêm trọng khả năng tương quan để phát hiện các chuỗi tấn công đa giai đoạn tinh vi.

Việc ứng dụng Mô hình Ngôn ngữ Lớn (LLM), đặc biệt là kiến trúc AI Tác tử (Agentic AI) và Retrieval-Augmented Generation (RAG), vào việc phân tích log tự động sinh ra những lỗ hổng chí mạng về cả vận hành lẫn bảo mật. Thứ nhất, độ trễ suy luận (reasoning latency) của hệ thống là quá cao đối với các môi trường luồng dữ liệu thời gian thực. Thứ hai, các kiến trúc Agentic AI mở rộng bề mặt tấn công (attack surface) vượt xa các LLM truyền thống. Các hệ thống tự trị này đặc biệt nhạy cảm với một phổ các véc-tơ tấn công tinh vi: các đòn *Indirect Prompt Injection* và *Jailbreaks* ẩn trong dữ liệu log [2], nơi kẻ tấn công chèn chỉ thị độc hại vào các trường động (ví dụ: User-Agent) nhằm thao túng logic của Agent. Hơn nữa, việc cấp quyền thực thi cho Agent tạo ra rủi ro *Thoát hộp cát* (*Sandbox Escape*) và *Thực thi mã từ xa* (*RCE*), trong khi việc hiển thị đầu ra của LLM trên bảng điều khiển mở ra lỗ hổng *Rò rỉ dữ liệu* (*Data Exfiltration*) thông qua render Markdown độc hại. Thêm vào đó, sự phụ thuộc vào cơ sở tri thức bên ngoài đặt hệ thống trước nguy cơ *Đầu độc tri thức* (*RAG Poisoning*), nơi kẻ xấu thay đổi tài liệu tham chiếu để bóp méo nhận thức ngữ cảnh của Agent. Điều này tạo ra một nghịch lý lớn: phân tích viên bắt buộc phải cấp cho Agent ngữ cảnh động và khả năng thực thi để điều tra các cuộc tấn công, nhưng việc này lại mở ra nguy cơ toàn bộ cơ sở hạ tầng SOC bị chiếm quyền điều khiển.

1.2 Mục tiêu Nghiên cứu

Nhận thấy những hạn chế trên, nghiên cứu này đặt mục tiêu phát triển một AI Security Agent tự trị dựa trên kiến trúc nhận thức hai tầng (cognitive two-tier architecture). Thay vì hoàn toàn phụ thuộc vào một LLM duy nhất để xử lý mọi log, luận văn đề xuất một kiến trúc hybrid lai: một động cơ quy tắc tốc độ cao kết hợp với một LLM Agent có khả năng suy luận sâu. Hướng tiếp cận này được kỳ vọng sẽ tối ưu hóa độ trễ suy luận bằng cách lọc nhiễu, giảm thiểu nguy cơ prompt injection qua cơ chế đóng gói có ranh giới, và nâng cao khả năng nhận

thức ngữ cảnh thông qua Bộ nhớ Rủi ro Dài hạn (Long-Term Threat Memory) và kiến trúc Dual-RAG. Một bảng điều khiển (Dashboard) kết hợp yếu tố Human-in-the-Loop (HITL) cũng sẽ được xây dựng để chứng minh tính thực tiễn của tự động hóa SOC.

1.3 Câu hỏi Nghiên cứu

Dựa trên các mục tiêu đã đề ra, luận văn tập trung giải quyết bốn câu hỏi nghiên cứu cốt lõi: 1. Việc áp dụng Kiến trúc Hai tầng (Rule-based + LLM Agent) kết hợp với Cơ sở Hành vi theo Phiên (Session-Aware Behavioral Baseline) có tối ưu hóa Độ trễ Suy luận so với phương pháp Một tầng (chỉ dùng LLM) hay không? 2. Liệu cơ chế Đóng gói Dữ liệu có Phân cách (Delimited Data Encapsulation) có thể ngăn chặn hiệu quả các đòn Prompt Injection mang tính cấu trúc (ví dụ: Delimiter Smuggling) mà không làm suy giảm độ chính xác phân loại của LLM? 3. Kiến trúc Dual-RAG (kết hợp khung chuẩn chiến thuật và kiểm soát phòng thủ) cải thiện Độ phù hợp Ngữ cảnh (Context Relevance) của quá trình phân tích rủi ro do LLM thực hiện như thế nào? 4. Một vòng lặp phản hồi (Feedback Loop) mang tính thích ứng với cơ chế Cách ly HITL (HITL Quarantine) có thể ngăn chặn Adversarial Rule Injection và đảm bảo an toàn cho các phản ứng tự động ra sao?

1.4 Đối tượng và Phương pháp Nghiên cứu

Đối tượng nghiên cứu chính bao gồm Kiến trúc Bảo mật Hai tầng, AI Tác tử (LLM ứng dụng trong tự động hóa SOC), các kỹ thuật tấn công Prompt Injection, và dữ liệu log phát hiện xâm nhập mạng. Nghiên cứu sử dụng phương pháp Thực nghiệm Định lượng, áp dụng thiết kế Thí nghiệm Loại bỏ (Ablation Study) để so sánh hệ thống Hai tầng với hệ thống cơ sở chỉ dùng LLM. Quá trình đánh giá được thực hiện qua Khung 5 Chiều (5D) bao trùm các chỉ số Phân loại, Độ trễ vận hành, Tính kháng lỗi Adversarial, Chất lượng Ngữ cảnh qua RAGAS (sử dụng LLM-as-a-Judge chéo họ), và Tính Giải thích.

1.5 Phạm vi Nghiên cứu

Nghiên cứu giới hạn trong phạm vi Phát hiện Xâm nhập Mạng (Network Intrusion Detection) và phân loại log tại SOC. Các thành phần trong phạm vi bao gồm: Pipeline luồng dữ liệu (Redis), Động cơ Quy tắc Tier-1, LangGraph Agent với Dual-RAG, Cơ chế Guardrails (Nén dữ liệu và Phòng thủ Injection), cùng Dashboard Streamlit hỗ trợ HITL với Bộ nhớ SQLite. Nghiên cứu không bao gồm việc tinh chỉnh (fine-tuning) các Mô hình Nền tảng (Foundation Models), triển khai hạ tầng phần cứng vật lý lớn, hay chống lại các đòn tấn công từ chối dịch vụ (DDoS) thể tích ở Layer 3/4.

1.6 Cấu trúc Luận văn

Luận văn được tổ chức thành sáu chương chính: (1) Giới thiệu và Bối cảnh; (2) Tổng quan Nghiên cứu; (3) Giải pháp Đề xuất và Phương pháp Nghiên cứu; (4) Kế hoạch Triển khai và Đánh giá; (5) Kết quả Kỳ vọng và Đóng góp Khoa học; và (6) Kết luận.

2. Tổng quan Nghiên cứu

2.1 Các Phương pháp Nền tảng

Trong giai đoạn đầu của lĩnh vực an ninh mạng, các Hệ thống Phát hiện Xâm nhập (IDS) chủ yếu dựa vào cơ chế đối sánh chữ ký (signature-based) (ví dụ: Snort, Suricata) để xác định các mối đe dọa đã biết. Mặc dù các hệ thống này mang lại hiệu suất cực cao về mặt thông lượng, chúng lại vấp phải những giới hạn nghiêm trọng trong việc nhận diện các đòn tấn công zero-day hoặc mã độc đa hình (polymorphic malware) [4].

Sự chuyển dịch sang Học máy (Machine Learning) đã mở ra kỷ nguyên của IDS dựa trên sự bất thường (anomaly-based), nơi các mô hình như Random Forest, LSTM và các transformer chuyên biệt như LogBERT [6] học cách xây dựng đường cơ sở hành vi bình thường và đánh dấu các sai lệch. Mặc dù các phương pháp này cải thiện khả năng phát hiện zero-day, chúng vẫn bị giới hạn cơ bản ở mức chấm điểm bất thường dựa trên mẫu. Chúng không thể giải thích *tại sao* một chuỗi log lại độc hại, không thể ánh xạ mối đe dọa vào các khung chuẩn, hay sinh ra các bước khắc phục có thể hành động — những khả năng đòi hỏi năng lực suy luận ngữ cảnh của Mô hình Ngôn ngữ Lớn. Về mặt thiết kế, các mô hình dựa trên khuôn mẫu này không có khả năng truy vấn các cơ sở tri thức bên ngoài trong quá trình suy luận—một hạn chế kiến trúc cốt lõi khiến chúng không thể lập bản đồ rủi ro theo ngữ cảnh và đưa ra các bước khắc phục khả thi.

Các hệ thống SIEM hiện đại ra đời để gom nhóm và tương quan log, song chúng vẫn đòi hỏi sức lực khổng lồ từ con người để chấp vá ngữ cảnh đầy đủ của một cuộc tấn công.

2.2 Các Công trình Liên quan

Sự bùng nổ của các Mô hình Ngôn ngữ Lớn (LLM) gần đây đã chỉ ra hướng đi mới trong việc tự động tóm tắt log và phản ứng sự cố. Các bài khảo sát toàn diện như [3] đã chứng minh xu hướng ngày càng tăng trong việc áp dụng các API LLM đám mây (ví dụ: GPT-4) cho tự động hóa SOC, bao gồm tóm tắt log và phân loại sự cố. Tuy nhiên, việc phụ thuộc vào API đám mây vi phạm các chính sách bảo mật dữ liệu nghiêm ngặt của tổ chức và tạo ra độ trễ mạng không thể chấp nhận cho phân loại thời gian thực.

Trong lĩnh vực phân tích mối đe dọa tăng cường tri thức, các phương pháp tiếp cận dựa trên đồ thị tri thức cho ánh xạ tình báo mối đe dọa [5] đã cho thấy kết quả khả quan trong việc tự động trích xuất và cấu trúc các kỹ thuật tấn công từ báo cáo CTI thành các đồ thị tuân thủ MITRE ATT&CK. Tuy nhiên, các nghiên cứu hiện tại phần lớn chỉ tập trung vào phân tích chiến thuật thuần túy, thiếu sự tích hợp với các tiêu chuẩn ứng phó sự cố vận hành (như

NIST SP 800-61r2) cần thiết cho việc khắc phục SOC có thể hành động.

Về vấn đề an toàn và tính kháng lỗi của bản thân LLM, danh sách OWASP Top 10 dành cho ứng dụng LLM [12] cùng các đánh giá chuyên sâu từ Agent Security Bench (ASB) [10] đã cảnh báo mức độ dễ tổn thương đáng báo động của các LLM Agent trước rủi ro prompt injection. Dù các thuật toán như Drain3 [11] rất phổ biến trong việc gom nhóm và nén log, chúng chủ yếu phục vụ mục đích lưu trữ chứ không mang thiết kế của một rào chắn an ninh chủ động. Hơn nữa, các phương pháp luận đánh giá LLM gần đây nhấn mạnh sự cần thiết phải giải quyết Thiên kiến Tự đề cao (Self-Enhancement Bias) bằng cách sử dụng các mô hình đánh giá chéo họ (cross-family evaluation) hoặc các phép kiểm định thống kê thay vì chỉ dựa vào mô hình LLM-as-a-Judge đơn lẻ [13, 14]. Bức tranh học thuật hiện tại cho thấy sự thiếu vắng của một kiến trúc thống nhất, có khả năng tự phòng vệ với quy trình đánh giá nghiêm ngặt, từ đó khẳng định tính cấp thiết và định hướng nghiên cứu của luận văn này.

2.2.1 Phân tích So sánh các Phương pháp Hiện có

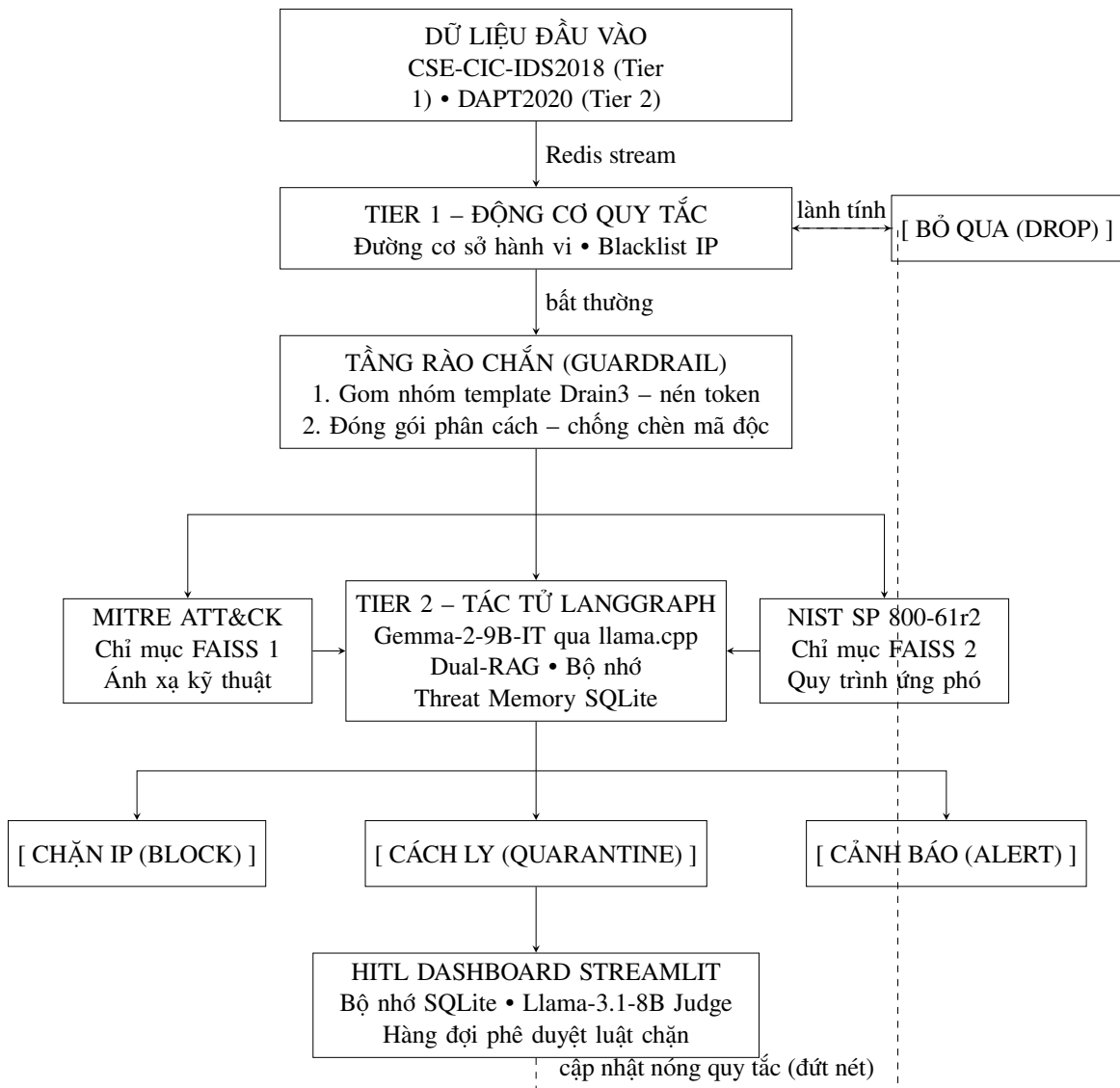
Bảng 1: So sánh các phương pháp phân tích log SOC

Phương pháp	Cục bộ	Thời gian thực	Chống Injection	RAG	HITL
Signature IDS (Snort)	✓	✓	N/A	—	—
ML Anomaly IDS (RF/LSTM)	✓	✓	—	—	—
LogBERT [6]	✓	✓	—	—	—
Cloud LLM SOC [3]	—	—	—	Partial	—
SENTINEL (Ours)	✓	✓	✓	✓	✓

3. Giải pháp Đề xuất và Phương pháp Nghiên cứu

3.1 Giải pháp Đề xuất: Kiến trúc Nhận thức Hai tầng

Giải pháp đề xuất dựa trên một Kiến trúc Nhận thức Hai tầng (Cognitive Two-Tier Architecture) được thiết kế với nguyên tắc Phân tách Mối quan tâm (Separation of Concerns) nghiêm ngặt. Cốt lõi của giải pháp là kết hợp tốc độ xử lý của các bộ quy tắc xác định (Tier 1) với năng lực suy luận sâu sắc của LLM Tác tử (Tier 2), được kết nối với nhau bởi các lớp rào chắn (guardrails) chống tấn công adversarial mạnh mẽ.



Hình 1: Kiến trúc Nhận thức Hai tầng

Tại tầng đầu tiên (Tier 1), Động cơ Quy tắc đóng vai trò như một màng lọc heuristic tốc độ cao. Nó duy trì các đường cơ sở (baselines) hành vi nhận thức theo phiên bằng cách

sử dụng cấu trúc dữ liệu trên bộ nhớ (như Redis) để theo dõi tần suất yêu cầu và các IP độc hại đã biết. Lớp này được thiết kế để sàng lọc phần lớn lưu lượng mạng lành tính hoặc nhiễu thể tích ngay tại tốc độ đường truyền (wire speed), với tỷ lệ lọc chính xác sẽ được đo lường thực nghiệm trong quá trình đánh giá. Điều này đảm bảo LLM ở tầng dưới không bị quá tải bởi lưu lượng khổng lồ.

Đối với những log có biểu hiện bất thường hoặc vượt qua ngưỡng mập mờ, chúng sẽ được chuyển đến Tầng Guardrail. Tại đây, quá trình vô hiệu hóa diễn ra theo hai bước tách biệt: 1. **Template Mining (Drain3)**: Trích xuất cấu trúc tĩnh của log để nén số lượng token mà không làm thay đổi các biến payload động. 2. **Đóng gói Dữ liệu có Phân cách (Delimited Data Encapsulation)**: Các trường payload động được bao bọc bên trong các dấu phân cách sinh ngẫu nhiên theo chuẩn mật mã học (ví dụ: <<RANDOM_HEX>>). Điều này ngăn chặn triệt để đòn tấn công Delimiter Smuggling, khiến kẻ thù không thể thoát khỏi ranh giới dữ liệu để thực thi Prompt Injection.

Sau khi vượt qua guardrail, dữ liệu log sạch sẽ tiến vào Tier 2, được quản lý bởi một Agent phát triển trên nền LangGraph. Agent này ứng dụng cơ chế Dual-RAG với hai chỉ mục FAISS riêng biệt, được truy vấn tuần tự và hợp nhất ngữ cảnh trước khi suy luận LLM:

- **Chỉ mục 1 — MITRE ATT&CK**: Ánh xạ các chỉ báo quan sát được vào các chiến thuật, kỹ thuật và thủ tục (TTP) cụ thể, trả lời câu hỏi *“cuộc tấn công nào đang xảy ra?”*
- **Chỉ mục 2 — NIST SP 800-61r2 [7]**: Truy xuất giai đoạn ứng phó sự cố phù hợp (Chuẩn bị, Phát hiện, Ngăn chặn, Loại bỏ, Khôi phục, Bài học kinh nghiệm), trả lời câu hỏi *“SOC cần làm gì tiếp theo?”*

Thiết kế hai chỉ mục cố định này loại bỏ nhu cầu về cơ chế định tuyến (router), giảm thiểu điểm lỗi và đảm bảo độ trễ có thể dự đoán cho quá trình đánh giá. Sự phân tách này bắc cầu giữa *nhận diện chiến thuật* (MITRE: chuyện gì đã xảy ra) và *phản ứng vận hành* (NIST: cần làm gì).

Mỗi chỉ mục sử dụng chiến lược **Tìm kiếm Lai (Hybrid Search)** kết hợp tìm kiếm ngữ nghĩa dày đặc (Dense Search qua FAISS với all-MiniLM-L6-v2) với tìm kiếm từ khóa thưa (Sparse Search qua BM25Okapi), được hợp nhất thông qua **Reciprocal Rank Fusion (RRF, k=60)** [15]. Cách tiếp cận tín hiệu kép này tối đa hóa cả khả năng hiểu ngữ nghĩa lẫn độ chính xác khớp từ khóa cho các định danh đặc thù an ninh mạng như CVE ID, địa chỉ IP và số cổng. Các đoạn văn bản được truy xuất sẽ trải qua quá trình **Làm sạch Cấu trúc (Structural Sanitization)**—chuẩn hóa Unicode, loại bỏ ký tự điều khiển và cắt giới hạn độ dài—nhằm phòng chống RAG Poisoning trước khi đưa vào prompt của LLM.

Cuối cùng, nhằm bảo vệ hệ thống trước đòn Adversarial Rule Injection, LLM không được phép trực tiếp thay đổi tường lửa ở Tier 1. Thay vào đó, các quy tắc do nó đề xuất sẽ bị đưa vào trạng thái Cách ly (Quarantine). Một quản trị viên SOC sẽ tiến hành rà soát các quy tắc này thông qua Dashboard (yếu tố Human-in-the-Loop) trước khi chúng được nạp nóng (hot-reload) vào hệ thống phòng thủ.

Bộ nhớ SQLite Threat Memory Ngắn hạn và Dài hạn: Tác tử Tier 2 dựa vào kho lưu trữ Threat Memory bằng SQLite bền vững để tương quan các cảnh báo an ninh mạng diễn ra trong nhiều ngày. Khi một log mới được leo thang lên, hệ thống sẽ truy vấn hồ sơ hoạt động lịch sử của IP đó. Nếu phát hiện các hành vi xảy ra thuộc nhiều giai đoạn khác nhau trong ma trận MITRE ATT&CK theo thời gian (ví dụ: Thăm dò mạng ở ngày 1, Xuyên nhập ban đầu ở ngày 2), mô-đun bộ nhớ sẽ tự động nâng mức cảnh báo của IP đó lên nguy cấp (HIGH/CRITICAL), cho phép phát hiện hiệu quả chuỗi APT tốc độ chậm (như đã chứng minh qua việc liên kết 197 chuỗi tấn công đa ngày từ bộ dữ liệu DAPT2020).

Bảng điều khiển Web Dashboard hỗ trợ HITL (SOC UI): Một giao diện quản trị chuyên nghiệp bằng Streamlit được xây dựng kèm phân quyền tài khoản (L1 Analyst và L3 Manager). Dashboard hiển thị thời gian thực hàng đợi cảnh báo, danh sách rule tường lửa đang chặn và mục quản trị cách ly. Streamlit Dashboard cũng hỗ trợ tính năng Điều tra Đối tượng Tương tác (Interactive Threat Investigation) cho phép chuyên gia SOC click chọn trực tiếp địa chỉ IP bất kỳ trên bảng để truy xuất ngay điểm uy tín rủi ro, lập luận của AI, thời điểm First/Last Seen và dòng thời gian (timeline) sự kiện lịch sử từ SQLite.

Theo dõi Thử nghiệm bằng MLflow: Nhằm phục vụ nghiên cứu thực nghiệm nghiêm túc, MLflow được tích hợp để tự động ghi nhật ký các chỉ số đánh giá của Ablation Study (F1-Score, Precision, Recall, FPR, độ trễ MTTR/MTTD, tỷ lệ trúng cache ngữ cảnh, tỷ lệ leo thang HITL) xuyên suốt các cấu hình khác nhau, hỗ trợ phân tích so sánh trực quan và tái lập thí nghiệm.

3.2 Tài liệu Kiến trúc Kỹ thuật Toàn diện

3.2.1 PHẦN 1: DATASET LAYER

Bước 1. CSE-CIC-IDS2018 Data Ingestion & Transformation

Tên bước: CSE-CIC-IDS2018 Data Ingestion & Transformation.

Tầng: Dataset Input Layer.

Mục đích: Biến đổi tập dữ liệu mạng tĩnh (CSV) thành dòng dữ liệu streaming thực tế mô phỏng môi trường mạng doanh nghiệp (Network Traffic Simulation), cung cấp đầu vào liên tục cho hệ thống.

Công nghệ sử dụng: pandas (ETL), Generator Python (yield), Sleep/Delay mechanics.

Cơ chế hoạt động: Đọc tệp CSV khổng lồ theo từng chunk (chunksize). Chuyển đổi các cột dữ liệu (Timestamp, Flow Duration, Bwd Packet Length Max...) thành JSON payload. Tạo độ trễ ngẫu nhiên (time.sleep) để mô phỏng Real-time Traffic/Wire speed.

Cấu trúc mã nguồn:

- File: scripts/fetch_and_build_dataset.py & src/streaming/publisher.py
- Hàm: stream_logs(), fetch_dataset()
- Đoạn mã logic:

```
for chunk in pd.read_csv("dataset.csv", chunksize=1000):
    for _, row in chunk.iterrows():
        payload = row.to_json()
        publish_to_redis(payload)
        time.sleep(random.uniform(0.01, 0.05))
```

- Input: Raw CSV/Parquet file → Output: JSON strings.

Luồng dữ liệu: CSV trên đĩa → Chunking Memory → JSON Formatting → Redis Pub/Sub channel.

Vai trò bảo mật: N/A — phục vụ hiệu năng và mô phỏng.

Bước 2. Multi-Stage APT Pattern Injection

Tên bước: Multi-Stage APT Pattern Injection.

Tầng: Dataset Input Layer.

Mục đích: Đảm bảo hệ thống có khả năng kiểm thử các chiến dịch Low-and-Slow APT kéo dài nhiều giai đoạn (Reconnaissance → Initial Access → Lateral Movement → Exfiltration), không chỉ các mã độc (malware) cục bộ.

Công nghệ sử dụng: JSON Serialization, HuggingFace Datasets, SQLite.

Cơ chế hoạt động: Load tập dữ liệu DAPT2020 có dán nhãn chuỗi tấn công (Kill-Chain). Các log được gán Session ID và giữ nguyên thứ tự thời gian tuyến tính để kích hoạt logic Long-Term Memory của Tier 2 sau này thông qua liên kết lịch sử.

Cấu trúc mã nguồn:

- File: scripts/fetch_dapt2020.py & scripts/build_dapt_chains.py
- Hàm: process_dapt2020(), build_chains()
- Input: DAPT2020 JSON → Output: Mapped Normalized JSON.

Luồng dữ liệu: Raw DAPT2020 logs → Data Normalizer (Đồng nhất schema với CIC-IDS) → Redis Channel.

Vai trò bảo mật: N/A — hỗ trợ đánh giá Contextual Correlation.

Bước 3. Redis Pub/Sub & Stateful Session Tracker

Tên bước: Redis Pub/Sub & Stateful Session Tracker.

Tầng: Dataset Input Layer / Middleware.

Mục đích: Hoạt động như một Message Broker tốc độ cao (In-memory) giữa Data Publisher và Tier 1 Rule Engine, đồng thời duy trì trạng thái (State) của các phiên kết nối mạng mà không cần Database lưu trữ vĩnh viễn.

Công nghệ sử dụng: redis-py, Redis Pub/Sub, Redis EXPIRE (TTL) / Local sliding memory.

Cơ chế hoạt động: Khi Publisher gửi log dạng JSON lên Redis Channel `sentinel_logs`, Subscriber lắng nghe. Với mỗi IP Nguồn mới, hệ thống tạo/cập nhật Session Profile trong `SessionBaseline`. Các log được liên kết phiên theo Source IP. Hệ thống thiết lập cơ chế tự dọn dẹp (eviction) các profile đã lâu không có traffic hoạt động vượt quá `ttl_seconds` để tránh cạn kiệt bộ nhớ.

Cấu trúc mã nguồn:

- File: `src/streaming/subscriber.py` & `src/tier1_filter/rule_engine.py`
- Lớp: `RedisSubscriber`, `SessionBaseline`
- Đoạn mã logic:

```
def _evict_stale_profiles(self):
    now = time.time()
    stale_ips = [ip for ip, profile in self.profiles.items()
                  if profile["last_seen"] and (now -
                                                profile["last_seen"]) > self.
                                                ttl_seconds]
    for ip in stale_ips:
        del self.profiles[ip]
```

- Input: JSON từ Pub/Sub → Output: In-memory Stateful IP Profiles.

Luồng dữ liệu: Network Stream → Redis Message Broker → Tier 1 Event Listener → Stateful Memory Update.

Vai trò bảo mật: Phòng chống tấn công từ chối dịch vụ (DoS) cạn kiệt tài nguyên bộ nhớ thông qua cơ chế tự động hủy Session rác (TTL/Eviction).

3.2.2 PHẦN 2: TIER 1 — RULE ENGINE

Bước 4. Dynamic Behavioral Baseline (Welford Outlier)

Tên bước: Unsupervised Outlier Detection using Welford's Algorithm.

Tầng: Tier 1 Rule Engine.

Mục đích: Xây dựng đường cơ sở (Baseline) hành vi mạng chuẩn cho từng IP theo thời gian thực và phát hiện zero-day attack dưới dạng các dị biệt thống kê phi giám sát mà không tốn tài nguyên bộ nhớ.

Công nghệ sử dụng: Thuật toán trực tuyến Welford ($O(1)$ time, $O(1)$ memory), Z-Score.

Cơ chế hoạt động: Duy trì các giá trị Trung bình (Mean) và Độ lệch chuẩn (Std Dev) chạy trực tuyến của các đặc trưng lưu lượng mạng chính. Khi có log mới, tính toán Z-Score: nếu $|Z| > 3.5$, đánh dấu là Outlier Anomaly và leo thang lên Tier 2 để điều tra.

Cấu trúc mã nguồn:

- File: `src/tier1_filter/rule_engine.py`
- Lớp: `RunningStats`, `RuleEngine`
- Đoạn mã logic:

```
def push(self, x: float):
    self.n += 1
    if self.n == 1:
        self.old_m = self.new_m = x
        self.old_s = 0.0
    else:
        self.new_m = self.old_m + (x - self.old_m) / self.n
        self.new_s = self.old_s + (x - self.old_m) * (x - self.new_m)
        self.old_m = self.new_m
        self.old_s = self.new_s
```

- Input: Flow statistics variables → Output: Z-Score / Escalation flag.

Luồng dữ liệu: Raw Flow variables → Welford calculations → Z-Score evaluation → Outlier Trigger.

Vai trò bảo mật: Phát hiện zero-day attack vượt qua các rule tĩnh truyền thống bằng cách kiểm soát độ lệch phân bố thống kê.

Bước 5. Static Threat Intelligence Lookup

Tên bước: Static Threat Intelligence Lookup.

Tầng: Tier 1 Rule Engine.

Mục đích: Drop tức thì (Wire-speed drop) các IP độc hại đã biết mà không cần phân tích tốn kém ở LLM.

Công nghệ sử dụng: $O(1)$ Hash Map (Python set hoặc Redis lookup).

Cơ chế hoạt động: Truy vấn IP nguồn (`src_ip`) vào một bảng Hash Table chứa danh sách IP xấu từ Threat Intel. Quá trình tra cứu mất chưa tới $O(1)$ thời gian. Nếu khớp, tăng điểm rủi ro và chặn lập tức.

Cấu trúc mã nguồn:

- File: `src/tier1_filter/rule_engine.py`
- Hàm: `_check_blacklist(ip)`
- Logic: `if ip in self.blacklist_set: return ThreatLevel.CRITICAL`
- Input: IP String → Output: Boolean (Matched/Unmatched).

Luồng dữ liệu: Extracted IP → Hash Table Memory Lookup → Drop Trigger.

Vai trò bảo mật: Chống lại Known Botnets, Mass Scanners, DDoS.

Bước 6. Heuristic Rate Limiting

Tên bước: Heuristic Rate Limiting.

Tầng: Tier 1 Rule Engine.

Mục đích: Chặn đứng các hành vi Brute-force, Syn Flood truyền thống bằng các rule cứng (Hard Rules).

Công nghệ sử dụng: Token Bucket / Leaky Bucket Heuristics (Python Variables).

Cơ chế hoạt động: Đếm số lượng request/packet vào các Port nhạy cảm (22 SSH, 3389 RDP) trong cửa sổ 1 giây hoặc 1 phút. Nếu vượt ngưỡng cho phép → Tính điểm rủi ro.

Cấu trúc mã nguồn:

- File: `src/tier1_filter/rule_engine.py`
- Hàm: `_evaluate_rate_limits()`
- Input: Flow Record → Output: Penalty Score.

Luồng dữ liệu: Port/Protocol extraction → Counter check → Scoring.

Vai trò bảo mật: Chống lại Brute-force Authentication, Port Scanning, DDoS.

Bước 7. Wire-speed Clean Traffic Dropping

Tên bước: Wire-speed Clean Traffic Dropping.

Tầng: Tier 1 Rule Engine.

Mục đích: Đây là màng lọc quan trọng nhất giúp Tier 2 (LLM Agent) không bị nghẽn (Bottleneck) thông qua việc triệt tiêu hơn 99% lượng log an toàn (Noise).

Công nghệ sử dụng: If-else Heuristic Logic (Python).

Cơ chế hoạt động: Nếu tổng điểm rủi ro (Risk Score) của một Session từ Baseline + Rate Limit + Blacklist nhỏ hơn một THRESHOLD_BENIGN (ví dụ: < 30 điểm), trả về hành động DROP. Log biến mất khỏi luồng xử lý và không được gửi đi.

Cấu trúc mã nguồn:

- File: `src/tier1_filter/rule_engine.py`
- Hàm: `evaluate()`
- Logic: `if total_score < self.risk_threshold: return {"tier1_action": "DROP"}`
- Input: Total Score → Output: Void / Drop Action.

Luồng dữ liệu: Scored Session → Garbage Collector (Hủy).

Vai trò bảo mật: Triệt tiêu cảnh báo giả và giảm tải tối đa cho mô hình ngôn ngữ lớn (Alert Fatigue Prevention).

Bước 8. Threat Escalation (Escalate to Tier 2)

Tên bước: Threat Escalation (Escalate to Tier 2).

Tầng: Tier 1 Rule Engine.

Mục đích: Đẩy các luồng mạng bất thường hoặc có biểu hiện outlier vượt ngưỡng ($|Z| > 3.5$) lên AI Agent để phân tích chuyên sâu.

Công nghệ sử dụng: Redis Queue, Message broker.

Cơ chế hoạt động: Khi `total_score >= THRESHOLD_ESCALATE` hoặc phát hiện dị biệt thống kê bởi Welford, Tier 1 đánh dấu hành động là ESCALATE và đẩy log vào bộ đệm của subscriber để gom thành batch gửi đến Tier 2.

Cấu trúc mã nguồn:

- File: `src/tier1_filter/rule_engine.py` & `src/streaming/subscriber.py`
- Hàm: `evaluate()`, `start_listening()`
- Input: Raw network log → Output: Escalate flag + Session history payload.

Luồng dữ liệu: Tier 1 evaluation → Escalate Flag → Subscriber batch buffer → LangGraph workflow input.

Vai trò bảo mật: Đưa các lưu lượng nghi ngờ, mập mờ vào luồng phân tích nhận thức sâu để phát hiện xâm nhập tinh vi.

3.2.3 PHẦN 3: GUARDRAIL LAYER

Bước 9. Log Parsing & Template Abstraction

Tên bước: Log Parsing & Template Abstraction.

Tầng: Guardrail Layer (Pre-processing).

Mục đích: Giải quyết bài toán giới hạn token của LLM, tránh việc đưa hàng nghìn dòng log giống hệt nhau làm tràn cửa sổ ngữ cảnh (Context Overflow).

Công nghệ sử dụng: drain3 library (Thuật toán Fixed Depth Tree).

Cơ chế hoạt động: Duyệt qua log mạng và tách các trường động (IP, Port, Timestamp) thành thẻ <*>. Xây dựng cây parse tree để gom nhóm các log có cùng cấu trúc tĩnh vào một Template ID và đếm tần suất xuất hiện của chúng.

Cấu trúc mã nguồn:

- File: src/guardrails/template_miner.py
- Lớp: LogTemplateMiner & TokenBudgetManager
- Hàm: add_log_dict()
- Input: Array of raw logs → Output: Dictionary of templates & occurrences.

Luồng dữ liệu: Raw JSON/string → Drain3 Tree Parser → Abstracted Template.

Vai trò bảo mật: Phòng chống tấn công làm cạn kiệt tài nguyên ngữ cảnh LLM (Context Exhaustion / DoS).

Bước 10. Summarized Token Budgeting

Tên bước: Summarized Token Budgeting.

Tầng: Guardrail Layer.

Mục đích: Đảm bảo dữ liệu gửi vào prompt luôn nằm trong giới hạn tối đa cho phép của mô hình (ví dụ 4000 tokens) nhưng vẫn giữ lại đầy đủ chỉ báo xâm nhập.

Công nghệ sử dụng: Tiktoken / LLM Tokenizer heuristics.

Cơ chế hoạt động: Đếm tổng số token của các template. Định dạng đầu ra dưới dạng Markdown cô đọng: [1000x] Template: Connection from IP <*>. Nếu vượt quá giới hạn token, hệ thống tự động cắt bỏ (truncate) các log ít quan trọng hơn.

Cấu trúc mã nguồn:

- File: src/guardrails/template_miner.py
- Lớp: TokenBudgetManager
- Hàm: fit_to_budget()
- Input: Abstracted Templates → Output: Compressed Markdown String.

Luồng dữ liệu: Template Dictionary → Length Evaluation → Truncated String.

Vai trò bảo mật: Tối ưu hóa hiệu năng và giảm chi phí suy luận (Inference cost).

Bước 11. Dynamic Prompt Encapsulation

Tên bước: Dynamic Prompt Encapsulation.

Tầng: Guardrail Layer.

Mục đích: Phân tách rạch ròi giữa chỉ thị hệ thống (System Prompt) và dữ liệu log đầu vào không đáng tin cậy. Ngăn LLM nhầm lẫn dữ liệu log là câu lệnh điều khiển.

Công nghệ sử dụng: Python `secrets.token_hex()` (Cryptographically Secure Pseudo-Random Number Generator).

Cơ chế hoạt động: Sinh ngẫu nhiên một chuỗi Delimiter có cấu trúc bảo mật (ví dụ: `===SENTINEL_DATA_` cho mỗi phiên. Hướng dẫn LLM chỉ được xử lý dữ liệu nằm giữa dấu phân cách này và bọc toàn bộ log vào bên trong.

Cấu trúc mã nguồn:

- File: `src/guardrails/prompt_filter.py`
- Lớp: `DelimitedDataEncapsulator`
- Đoạn mã logic:

```
delimiter = f"===SENTINEL_DATA_{secrets.token_hex(8)}=== "  
return f"{delimiter}\n{raw_data}\n{delimiter}"
```

- Input: Compressed String → Output: Encapsulated String.

Luồng dữ liệu: Compressed Log → Hex Generator → Wrapped Payload.

Vai trò bảo mật: Phòng chống tấn công chèn mã lệnh trực tiếp (Direct Prompt Injection).

Bước 12. Delimiter Smuggling Prevention

Tên bước: Delimiter Smuggling Prevention.

Tầng: Guardrail Layer.

Mục đích: Ngăn chặn kẻ tấn công tự sinh chuỗi phân cách giả mạo (ví dụ chèn `===SENTINEL_DATA_` trong User-Agent payload) để đóng ngoặc sớm và chèn lệnh điều khiển bên ngoài vùng đóng gói.

Công nghệ sử dụng: Regex Pattern Matching.

Cơ chế hoạt động: Quét qua dữ liệu log thô bằng Regex trước khi đóng gói. Nếu phát hiện bất kỳ chuỗi nào trùng khớp với định dạng Delimiter, hệ thống lập tức loại bỏ (`[DELIMITER_STRIPPED]`) để vô hiệu hóa nỗ lực smuggling.

Cấu trúc mã nguồn:

- File: `src/guardrails/prompt_filter.py`
- Lớp: `DelimitedDataEncapsulator`
- Hàm: `_strip_smuggling_attempts()`
- Input: Raw data string, Delimiter Pattern → Output: Cleaned data string.

Luồng dữ liệu: Raw string → Regex Stripper → Safe String.

Vai trò bảo mật: Phòng chống tấn công Delimiter Smuggling / Escape Injection.

Bước 13. Structural Sanitization & Cache Hardening

Tên bước: Structural Sanitization & Cache Hardening.

Tầng: Guardrail Layer (Output/Input Sanitizer).

Mục đích: Bảo vệ công cụ RAG khỏi nhiều ngữ nghĩa (Semantic Confusion), giải mã các obfuscation của hacker, cấm XSS/Markdown Exfiltration khi hiển thị, và trung hòa các mã độc tiềm ẩn lấy ra từ Semantic Cache (Cache Hit path).

Công nghệ sử dụng: Unicode Normalization (NFKC), Base64/Hex decoding, HTML/Markdown Sanitization, RAG Cache Sanitizer.

Cơ chế hoạt động: Giải mã các payload Hex/Base64 độc hại, loại bỏ ký tự zero-width và control characters. Quét loại bỏ các thẻ HTML nguy hiểm và cú pháp hình ảnh Markdown (!alt) để cấm LLM đẩy dữ liệu nhạy cảm ra ngoài qua render HTML. Đồng thời, khi đọc dữ liệu từ Semantic Cache, phương thức `sanitize_cache_entry()` sẽ làm sạch đệ quy kết quả và ngữ cảnh để tránh đầu độc bộ nhớ đệm (Semantic Cache Poisoning).

Cấu trúc mã nguồn:

- File: `src/guardrails/rag_sanitizer.py` & `src/rag/retriever.py`
- Lớp: `RAGSanitizer`
- Function: `sanitize_ingest()`, `sanitize_cache_entry()`
- Input: LLM Output / Log Input / Cache Entry → Output: Sanitized String / Sanitized Cache Dict.

Luồng dữ liệu: Raw text / Cache Hit Data → Normalization → `RAGSanitizer` Filtering → Sanitized Context.

Vai trò bảo mật: Phòng chống RAG Poisoning, Semantic Cache Poisoning, XSS, và rò rỉ dữ liệu qua giao diện quản trị (Data Exfiltration).

3.2.4 PHẦN 4: TIER 2 — LANGGRAPH AGENT

Bước 14. Cognitive Workflow Orchestration

Tên bước: Cognitive Workflow Orchestration.

Tầng: Tier 2 LangGraph Agent.

Mục đích: Xây dựng cỗ máy trạng thái hữu hạn (FSM) luân chuyển dữ liệu qua các node xử lý một cách an toàn, duy trì trạng thái của phiên điều tra mà không làm mất thông tin.

Công nghệ sử dụng: `langgraph.graph.StateGraph`, `TypedDict` (State Schema).

Cơ chế hoạt động: Định nghĩa cấu trúc `SentinelState` mang thông tin phiên điều tra. Đăng ký các nút xử lý: `rag_context` → `llm_triage` → `execute_action`.

Tại `llm_triage`, kết quả phân loại từ LLM sẽ kích hoạt Conditional Edges để điều hướng hành động tiếp theo.

Cấu trúc mã nguồn:

- File: `src/agent/workflow.py` & `src/agent/nodes.py`
- Class/Function: `SentinelState`, `create_agent_workflow()`, `route_triage_decision`
- Đoạn mã logic:

```
workflow.add_node("rag_context", node_rag_context)
workflow.add_node("llm_triage", node_llm_triage)
workflow.add_edge("rag_context", "llm_triage")
workflow.add_conditional_edges("llm_triage",
                               route_triage_decision)
```

- Input: State Object → Output: Compiled Graph App.

Luồng dữ liệu: Raw Incident → Guardrails Node → RAG Node → LLM Node → Action/HITL Node.

Vai trò bảo mật: Ràng buộc quy trình logic bảo mật (Logic Enforcer), cấm tác tử tự động thực thi phản hồi mạng mà không qua các trạm thẩm định.

Bước 15 & 16. FAISS Index 1 & 2 (MITRE ATT&CK & NIST SP 800-61r2)

Tên bước: FAISS Index 1 & 2 (MITRE ATT&CK & NIST SP 800-61r2).

Tầng: Tier 2 LangGraph Agent (RAG Sub-module).

Mục đích: Cung cấp tri thức chuyên sâu về kỹ thuật tấn công (MITRE ATT&CK) và quy trình ứng phó (NIST) dựa trên ngữ nghĩa của log mạng, ngăn chặn ảo giác (Hallucination) của LLM và đảm bảo tính toàn vẹn của tri thức tĩnh.

Công nghệ sử dụng: `faiss-cpu`, mô hình embedding `all-MiniLM-L6-v2` (384 chiều), SHA-256 verification.

Cơ chế hoạt động: Tài liệu MITRE và NIST được lưu dưới dạng vector 384 chiều trong 2 chỉ mục FAISS. Khi có truy vấn từ log, hệ thống mã hóa nó thành vector và tìm kiếm top-K văn bản có khoảng cách Euclidean nhỏ nhất. Đặc biệt, trước khi nạp cơ sở tri thức, hàm `verify_document_integrity()` trong `security.py` tự động kiểm tra chữ ký mã băm SHA-256 của toàn bộ tệp tài liệu nguồn đối chiếu với tệp `checksums.sha256` để phát hiện và ngăn chặn triệt để hành vi can thiệp trái phép.

Cấu trúc mã nguồn:

- File: `src/rag/embedder.py` & `src/rag/retriever.py` & `src/rag/security.py`
- Class: `DualRetriever`
- Function: `_retrieve_dense(query, index)`, `verify_document_integrity()`

- Input: Query String → Output: List of Top-K Document Dicts.

Luồng dữ liệu: Query String → Embedding Model → Vector → FAISS L2 Search → Ranked Chunks.

Vai trò bảo mật: Đảm bảo LLM lập luận bảo mật dựa trên các khung tiêu chuẩn chính thống, giảm dương tính giả. Đồng thời, bảo đảm tính toàn vẹn của tri thức tính chống lại các tấn công RAG Poisoning.

Bước 17. BM25Okapi Sparse Keyword Matching

Tên bước: BM25Okapi Sparse Keyword Matching.

Tầng: Tier 2 LangGraph Agent (RAG Sub-module).

Mục đích: Khắc phục điểm yếu của dense vector search trong việc tìm kiếm các từ khóa chính xác như CVE ID, địa chỉ IP hoặc port đặc thù.

Công nghệ sử dụng: `rank_bm25.BM25Okapi`.

Cơ chế hoạt động: Kho tài liệu được tokenize và lập chỉ mục ngược. Điểm số tương đồng từ khóa được tính toán bằng thuật toán BM25Okapi để lọc ra các văn bản chứa chính xác định danh bảo mật cần tìm.

Cấu trúc mã nguồn:

- File: `src/rag/retriever.py`
- Class: `DualRetriever` (khởi tạo `bm25_mitre` và `bm25_nist`)
- Function: `_retrieve_sparse(query_tokens)`
- Input: Query Tokens → Output: List of Sparse Scores.

Luồng dữ liệu: Query Tokens → BM25 calculation → Ranked Sparse Chunks.

Vai trò bảo mật: N/A — Cải thiện độ chính xác tìm kiếm RAG đối với các chỉ báo xâm nhập cụ thể.

Bước 18. Reciprocal Rank Fusion (RRF, k=60) Score Merging

Tên bước: Reciprocal Rank Fusion (RRF, k=60) Score Merging.

Tầng: Tier 2 LangGraph Agent (RAG Sub-module).

Mục đích: Hợp nhất kết quả xếp hạng từ FAISS (Dense) và BM25 (Sparse) một cách công bằng mà không cần chuẩn hóa điểm số khác biệt về thang đo của 2 thuật toán.

Công nghệ sử dụng: Thuật toán Reciprocal Rank Fusion (RRF) với hằng số $k = 60$.

Cơ chế hoạt động: Chạy Dense và Sparse song song. Lấy vị trí thứ hạng (rank) của từng tài liệu trong 2 list và cộng điểm RRF: $RRF_Score = \sum \frac{1}{k+rank}$. Sắp xếp lại danh sách theo điểm số này.

Cấu trúc mã nguồn:

- File: `src/rag/retriever.py`
- Function: `_rrf_merge(dense_results, sparse_results, k=60)`
- Đoạn mã logic:

```
rrf_scores[doc_id] += 1.0 / (k + rank)
```

- Input: 2 Ranked Lists → Output: 1 Final Sorted Top-K List.

Luồng dữ liệu: Dense Ranks + Sparse Ranks → RRF merging → Final Sorted Context.

Vai trò bảo mật: N/A — Đảm bảo ngữ cảnh đưa vào LLM là tối ưu nhất.

Bước 19. Context Merge and LLM Prompt Construction (Active Learning)

Tên bước: Context Merge and LLM Prompt Construction (Active Learning).

Tầng: Tier 2 LangGraph Agent.

Mục đích: Tạo lập prompt hoàn chỉnh chứa logs mạng, tri thức RAG và bổ sung thêm các ví dụ thực tiễn từ vòng lặp Few-shot Active Learning để LLM tự học quyết định sửa sai của con người.

Công nghệ sử dụng: Jinja2 / Prompt formatting.

Cơ chế hoạt động: Đọc các quyết định Approve/Reject trước đây của analyst trong cấu hình hệ thống, sinh các ví dụ few-shot đưa trực tiếp vào cuối System Prompt để định hình lập luận bảo mật cho LLM.

Cấu trúc mã nguồn:

- File: `src/agent/prompts.py` & `src/agent/nodes.py`
- Function: `build_triage_prompt(log_data, rag_context)`
- Input: Logs + Context → Output: Message Array [{"role": "system", ...}, {"role": "user", ...}]

Luồng dữ liệu: Sub-components + Active Learning examples → Prompt Builder → LLM Input.

Vai trò bảo mật: Giúp tác tử tự sửa sai và cải thiện độ chính xác phân loại theo thời gian dựa trên tương tác con người.

Bước 20. Gemma-2-9B-IT Inference via llama.cpp

Tên bước: Gemma-2-9B-IT Inference via llama.cpp.

Tầng: Tier 2 LangGraph Agent.

Mục đích: "Bộ não" chính của toàn hệ thống, chịu trách nhiệm đọc ngữ cảnh, suy luận logic an ninh mạng và đưa ra phán quyết.

Công nghệ sử dụng: Server `llama.cpp`, mô hình cục bộ Gemma-2-9B-IT định dạng GGUF lượng tử hóa 4-bit (Q4_K_M).

Cơ chế hoạt động: Phục vụ mô hình hoàn toàn offline trên GPU (RTX 4060 Ti). Gửi HTTP POST request tới API tương thích OpenAI của server llama.cpp. Cấu hình `temperature=0.1` để đảm bảo câu trả lời mang tính xác định cao nhất và tuân thủ định dạng JSON.

Cấu trúc mã nguồn:

- File: `src/agent/llm_client.py`
- Class: `OpenAILLMClient`
- Function: `invoke(messages, temperature)`
- Input: Message Array → Output: JSON String.

Luồng dữ liệu: Node Triage → HTTP POST Request → GPU Compute → JSON Response.

Vai trò bảo mật: Thực hiện phân tích nhận thức sâu để phát hiện các mẫu tấn công tinh vi.

Bước 21. Decision Output: BLOCK / QUARANTINE / ALERT

Tên bước: Decision Output: BLOCK / QUARANTINE / ALERT.

Tầng: Tier 2 LangGraph Agent.

Mục đích: Biến kết quả phán quyết JSON của LLM thành các hành động tác động vật lý lên tường lửa hoặc đưa vào quarantine.

Công nghệ sử dụng: Python API Callbacks, OS Firewall command scripting.

Cơ chế hoạt động: Đọc trường `action` từ JSON: - `BLOCK_IP`: Gọi hàm chặn IP ở mức mạng và ghi nhận quy tắc vào hàng đợi cách ly chờ phê duyệt. - `ALERT`: Ghi cảnh báo đỏ lên DB mà không cấm. - `AWAIT_HITL (QUARANTINE)`: Phán quyết mập mờ, chuyển thẳng vào hàng đợi chờ analyst duyệt.

Cấu trúc mã nguồn:

- File: `src/agent/nodes.py` & `src/response/executor.py`
- Function: `node_action_executor(), block_ip()`
- Input: Parsed JSON Dictionary → Output: Action Executions.

Luồng dữ liệu: LLM Output JSON → Callback Router → Execution / Database write.

Vai trò bảo mật: Cô lập tấn công, phản ứng ngăn chặn kịp thời (Threat Neutralization).

Bước 22 & 23. Long-Term Threat Memory & APT Chain Tracking Logic

Tên bước: Long-Term Threat Memory & APT Chain Tracking Logic.

Tầng: Tier 2 LangGraph Agent (Memory).

Mục đích: Lưu trữ uy tín (Reputation) dài hạn của các IP vi phạm và kết nối các hành vi đơn lẻ qua nhiều ngày để phát hiện các chiến dịch APT low-and-slow.

Công nghệ sử dụng: Cơ sở dữ liệu SQLite, IP Reputation Scoring.

Cơ chế hoạt động: Lưu vết mọi incident vi phạm vào SQLite. Khi có IP mới bị leo thang, hệ thống tra cứu SQLite xem IP này đã thực hiện các kỹ thuật gì. Nếu IP vi phạm liên quan đến ≥ 3 giai đoạn khác nhau trong ma trận MITRE ATT&CK theo thời gian → Đánh dấu cảnh báo APT nguy cấp và nâng mức cảnh báo của IP lên critical.

Cấu trúc mã nguồn:

- File: `src/agent/threat_memory.py`
- Class: `ThreatMemoryManager`
- Function: `record_incident()`, `check_apt_pattern()`
- Input: IP, Action, MITRE ID → Output: SQLite Insert/Update, APT Candidate Flag (Boolean).

Luồng dữ liệu: Action Node → SQLite Write → (Chu kỳ sau) SQLite Read → System Prompt.

Vai trò bảo mật: Phòng chống tấn công APT có kỹ năng ẩn mình và đổi chiến thuật qua thời gian dài để bypass Signature.

3.2.5 PHẦN 5: HITL DASHBOARD

Bước 24. Streamlit UI Layout, RBAC & Authentication Hardening

Tên bước: Streamlit UI Layout, RBAC & Authentication Hardening.

Tầng: HITL Dashboard Layer.

Mục đích: Cung cấp giao diện trực quan thời gian thực (Real-time GUI) cho phép chuyên gia an ninh mạng theo dõi hệ thống, điều tra sự cố, phê duyệt quy tắc một cách an toàn và bảo mật cao.

Công nghệ sử dụng: `streamlit`, `streamlit-autorefresh`, `st.session_state`, `PBKDF2-HMAC-SHA256`, `constant-time compare`.

Cơ chế hoạt động: Tổ chức giao diện dưới dạng các Tabs chức năng. Tích hợp màn hình xác thực phân quyền (RBAC) cho Analyst (L1) và Manager (L3). Để chống tấn công Auth Input Injection và Brute-force, Username được lọc chặt chẽ qua biểu thức chính quy `^[a-zA-Z0-9_]{1,30}$`, mật khẩu được băm bằng thuật toán an toàn PBKDF2-HMAC-SHA256 kết hợp `constant-time comparison` để loại bỏ rủi ro tấn công kênh kề (Side-channel Timing Attacks).

Cấu trúc mã nguồn:

- File: `src/ui/app.py` & `src/ui/auth.py`
- Function: `main_dashboard()`, `login_screen()`, `hash_password()`
- Input: User credentials → Output: Session State / GUI.

Luồng dữ liệu: Input Credentials → Regex validation → PBKDF2 Hashing → constant-time comparison → `st.session_state` configuration.

Vai trò bảo mật: Phòng ngừa các đòn tấn công chèn mã Form Input Injection, side-channel timing attacks, và kiểm soát truy cập phân quyền nghiêm ngặt (Zero Trust HITL Auth).

Bước 25. Quarantine Rule Review & SOC Manager Workflow

Tên bước: Quarantine Rule Review & SOC Manager Workflow.

Tầng: HITL Dashboard Layer.

Mục đích: Ngăn chặn LLM tự động đưa ra các quyết định cấm nhảm IP quan trọng hoặc gateway bằng cách bắt buộc phải có sự xác nhận của quản trị viên (L3 Manager).

Công nghệ sử dụng: RBAC Session, Streamlit Buttons.

Cơ chế hoạt động: Các quy tắc chặn do AI đề xuất sẽ ở trạng thái PENDING và đưa vào hàng đợi cách ly. Giao diện chỉ cho phép tài khoản có vai trò manager (L3) click duyệt (**Approve**) hoặc từ chối (**Reject**).

Cấu trúc mã nguồn:

- File: `src/ui/app.py` & `src/tier1_filter/feedback_listener.py`
- Function: `approve_rule()`, `get_active_dynamic_rules()`
- Input: Manager Click Event → Output: Rule status change (PENDING → ACTIVE/REJECTED)

Luồng dữ liệu: UI Click → State change → Ghi file cấu hình `system_settings.yaml`.

Vai trò bảo mật: Phòng chống tấn công Adversarial Rule Injection.

Bước 26. Live Production False Positive Rate (FPR) Tracker

Tên bước: Live Production False Positive Rate (FPR) Tracker.

Tầng: HITL Dashboard Layer.

Mục đích: Trực quan hóa tỷ lệ cảnh báo sai thực tế của mô hình LLM tại môi trường vận hành thời gian thực để chuyên gia SOC đánh giá hiệu năng AI.

Công nghệ sử dụng: Streamlit Metric Cards, Python calculations.

Cơ chế hoạt động: Hệ thống tính toán tỷ lệ cảnh báo sai dựa trên phản hồi bác bỏ của analyst:
$$\text{Live FPR} = \frac{\text{Rejected Rules}}{\text{Approved} + \text{Rejected}} \times 100\%.$$
 Thẻ KPI hiển thị màu sắc Neon để chỉ thị cảnh báo tương ứng (Xanh lá < 10

Cấu trúc mã nguồn:

- File: `src/ui/app.py` & `src/ui/components.py`
- Function: `render_metrics_header()`
- Input: Configuration count lists → Output: Live FPR Score (Float).

Luồng dữ liệu: Rule counters → FPR Formula → Rendered Dashboard Metric Card.

Vai trò bảo mật: Đánh giá tính kháng nhiễu và độ tin cậy thực tế của bộ óc AI trong môi trường sản xuất.

Bước 27. Docker Model Switcher Orchestration

Tên bước: Docker Model Switcher Orchestration.

Tầng: HITL Dashboard Layer.

Mục đích: Cho phép quản trị viên chuyển đổi nhanh cấu hình hoặc hoán đổi mô hình LLM đang chạy (ví dụ giữa Gemma và Llama) trực tiếp từ giao diện mà không gây gián đoạn luồng xử lý chính.

Công nghệ sử dụng: Bash scripting, Docker-compose env hot-reload.

Cơ chế hoạt động: Khi admin chọn model trên UI và bấm áp dụng, hệ thống gọi script chạy ngầm cập nhật biến môi trường LLM_MODEL_FILE trong file .env và restart container sentinel_llm tự động.

Cấu trúc mã nguồn:

- File: scripts/switch_model.sh & src/ui/app.py
- Input: Model name string → Output: Environment hot-reload.

Luồng dữ liệu: UI Select → Script execution → Docker Container Restart → Port 5000 API reconnect.

Vai trò bảo mật: Hỗ trợ tính linh hoạt và dự phòng lỗi (Failover) của bộ óc AI.

Bước 28. HMAC Cryptographic Log Chaining

Tên bước: HMAC Cryptographic Log Chaining.

Tầng: HITL Dashboard Layer / Response.

Mục đích: Chống giả mạo nhật ký sự kiện trong DB SQLite bằng cách tạo ra một chuỗi liên kết băm mật mã học (HMAC Log Chaining) tương tự cấu trúc Blockchain.

Công nghệ sử dụng: hmac (SHA-256), sqlite3.

Cơ chế hoạt động: Khi ghi một dòng log mới vào bảng audit_trail, giá trị băm HMAC được tính toán dựa trên nội dung log và hash của dòng log trước đó. Nút kiểm tra tính toàn vẹn logs quét qua DB, tính toán lại hash và báo đỏ nếu phát hiện bất kỳ sự thay đổi dữ liệu trái phép nào.

Cấu trúc mã nguồn:

- File: src/response/executor.py & src/ui/app.py
- Function: _log_to_db(), verify_log_chain()
- Input: Log record data → Output: Cryptographic Hash string.

Luồng dữ liệu: Log data → HMAC calculation → SQLite Write → Integrity Check.

Vai trò bảo mật: Phòng chống tấn công xâm nhập DB sửa xóa dấu vết log (Non-repudiation / Forensics Protection).

Bước 29 & 30. Cross-family Judge Invocation & RAGAS Metric Computation

Tên bước: Cross-family Judge Invocation & RAGAS Metric Computation.

Tầng: Evaluation Framework / HITL Context.

Mục đích: Chấm điểm tự động và độc lập chất lượng ngữ cảnh truy xuất RAG của Gemma-2-9B-IT mà không bị ảnh hưởng bởi thiên kiến tự đề cao.

Công nghệ sử dụng: Mô hình Llama-3.1-8B-Instruct (trọng tài), RAGAS-inspired Prompting Rubrics.

Cơ chế hoạt động: Sử dụng mô hình Llama-3.1 (khác họ với Gemma) làm trọng tài chấm điểm câu trả lời từ 0.0 đến 1.0 cho 4 khía cạnh: Context Precision, Answer Relevancy, Faithfulness, Context Recall dựa trên các tiêu chí RAGAS.

Cấu trúc mã nguồn:

- File: `experiments/evaluate_reasoning.py`
- Function: `evaluate_with_llama_judge()`
- Input: Gemma Output + Context → Output: 4 Metric Scores (JSON).

Luồng dữ liệu: RAG Output + Gemma Response → Llama-3.1 Prompt → Metric Scores.

Vai trò bảo mật: Đánh giá chất lượng và độ trung thực của RAG, chống ngộ độc tri thức (RAG Poisoning).

3.2.6 PHẦN 6: EVALUATION FRAMEWORK (5D)

Bước 31. Ablation Study Setup

Tên bước: Ablation Study Setup.

Tầng: Evaluation Framework.

Mục đích: Chứng minh hiệu năng của kiến trúc 2-Tier thông qua thử nghiệm loại bỏ (Ablation Study) bằng cách so sánh 6 cấu hình từ A đến F.

Công nghệ sử dụng: YAML configs, MLflow Experiment Tracking.

Cơ chế hoạt động: Bật/tắt các cấu phần hệ thống thông qua các file YAML. Chạy thử nghiệm tự động trên tập dữ liệu ground truth và ghi nhật ký chỉ số lên MLflow để phân tích trực quan.

Cấu trúc mã nguồn:

- File: `config/ablation/*.yaml & experiments/run_ablation_study.py`

- Input: YAML Config → Output: MLflow logs.

Luồng dữ liệu: Config YAML → Ablation script run → Metric results → MLflow dashboard.

Vai trò bảo mật: N/A — Phương pháp luận thực nghiệm nghiên cứu khoa học.

Bước 32. Classification Metrics & McNemar's Test

Tên bước: Classification Metrics & McNemar's Test.

Tầng: Evaluation Framework.

Mục đích: Đo lường khả năng phân loại và kiểm định ý nghĩa thống kê của việc cải thiện độ chính xác (Precision, Recall, F1, FPR).

Công nghệ sử dụng: `scikit-learn`, `statsmodels` (McNemar's Test).

Cơ chế hoạt động: So sánh nhãn dự đoán và ground truth trên 750 mẫu. Chạy kiểm định McNemar trên ma trận nhầm lẫn để tính toán p-value: nếu $p < 0.05$, chứng minh cải tiến có ý nghĩa thống kê thực tế.

Cấu trúc mã nguồn:

- File: `experiments/statistical_tests.py` & `run_ablation_study.py`
- Function: `compute_classification_metrics()`, `mcnemar_test()`
- Input: Prediction arrays → Output: Quality scores & p-value.

Luồng dữ liệu: Result arrays → Sklearn formulas → P-value.

Vai trò bảo mật: N/A — Đánh giá tính chính xác tổng thể trong phát hiện xâm nhập.

Bước 33. Operational Metrics & Mann-Whitney U Test

Tên bước: Operational Metrics & Mann-Whitney U Test.

Tầng: Evaluation Framework.

Mục đích: Đo lường độ trễ xử lý (MTTR/MTTD proxy), tỷ lệ leo thang và cache hit rate để chứng minh tính khả thi triển khai trong thực tế.

Công nghệ sử dụng: Python `time`, `scipy.stats` (Mann-Whitney U Test).

Cơ chế hoạt động: Ghi nhận thời gian suy luận. So sánh độ trễ giữa hệ thống 1-tier và 2-tier bằng kiểm định phi tham số Mann-Whitney U để xác minh việc giảm 99.8

Cấu trúc mã nguồn:

- File: `experiments/statistical_tests.py`
- Function: `mann_whitney_u_test(latency_A, latency_F)`
- Input: Latency arrays → Output: U-test p-value.

Luồng dữ liệu: Time Deltas → U-test math → p-value.

Vai trò bảo mật: Xác thực hiệu năng của màng lọc Tier 1 trong việc chống nguyền LLM.

Bước 34. Robustness against Adversarial Attacks

Tên bước: Robustness against Adversarial Attacks.

Tầng: Evaluation Framework.

Mục đích: Đánh giá độ bền bỉ và khả năng phòng thủ của hệ thống trước 45 mẫu tấn công nghịch đảo tinh vi.

Công nghệ sử dụng: JSON adversarial dataset (45 mẫu).

Cơ chế hoạt động: Nhồi các log chứa chuỗi prompt injection và encoding bypass đặc chế. Tính toán tỷ lệ phần trăm payload có thể bypass được guardrail để thao túng LLM.

Cấu trúc mã nguồn:

- File: `experiments/evaluate_robustness.py`
- Function: `run_robustness_eval()`
- Input: Adversarial JSON List → Output: Defeat Rate Percentage.

Luồng dữ liệu: Adversarial logs → Guardrails pipeline → Block/Pass decisions.

Vai trò bảo mật: Đo lường khả năng chống đỡ trước mã độc chèn prompt injection ẩn giấu trong log.

Bước 35. Explainability (Audit Trail Completeness Rate)

Tên bước: Explainability (Audit Trail Completeness Rate).

Tầng: Evaluation Framework.

Mục đích: Đo lường tính giải thích được (Explainability) của mô hình một cách xác định thông qua kiểm tra cấu trúc dữ liệu bắt buộc.

Công nghệ sử dụng: Python Key/Field Validation Logic.

Cơ chế hoạt động: Quét các câu trả lời JSON của LLM và kiểm tra sự hiện diện đầy đủ của 5 trường dữ liệu quan trọng: `action`, `confidence`, `reasoning`, `target`, và `mitre_technique`. Tính toán tỷ lệ phần trăm hoàn thành.

Cấu trúc mã nguồn:

- File: `experiments/evaluate_reasoning.py`
- Function: `check_audit_completeness()`
- Logic: `valid = all(k in response for k in REQUIRED_FIELDS)`
- Input: LLM JSON response → Output: Completeness percentage.

Luồng dữ liệu: Output JSON → Key validator → Completeness metric.

Vai trò bảo mật: Đảm bảo hệ thống đạt chuẩn Explainable AI (XAI) cho ứng cứu sự cố SOC, tránh hiện tượng hộp đen AI.

3.3 Công nghệ và Công cụ

Hệ thống được xây dựng hoàn toàn trên các framework Python hiện đại. Quá trình xử lý luồng và theo dõi phiên ngắn hạn dựa vào Redis. Động cơ suy luận LLM sử dụng mô hình cục bộ đã được lượng tử hóa (Gemma-2-9B-IT) ở định dạng GGUF Q4_K_M, được phục vụ thông qua llama.cpp, đảm bảo quyền riêng tư dữ liệu tuyệt đối và khả năng vận hành hoàn toàn ngoại tuyến.

Luồng công việc của Agent được điều phối bởi LangGraph, cho phép ra quyết định tuần hoàn và sử dụng công cụ. Đối với phần truy xuất tri thức, thư viện `sentence-transformers` kết hợp với FAISS được sử dụng cho tìm kiếm vector ngữ nghĩa dày đặc (Dense Search) và BM25Okapi (`rank_bm25`) cho tìm kiếm từ khóa thưa (Sparse Search). Giao diện người dùng và bảng điều khiển HITL được phát triển bằng Streamlit, trong khi bộ nhớ rủi ro dài hạn và nhật ký kiểm toán (audit trails) được lưu trữ bền vững qua SQLite.

3.3.1 Cấu hình Phần cứng

Để đảm bảo tính tái lập hoàn toàn của các phép đo hiệu năng, toàn bộ thí nghiệm được thực hiện trên cấu hình phần cứng sau:

Thành phần	Thông số
CPU	Intel Core i7-14700KF (28 luồng) @ 5.700 GHz
GPU	NVIDIA GeForce RTX 4060 Ti (16 GB VRAM)
RAM	32 GB DDR5
HĐH	Ubuntu 24.04.4 LTS (x86_64, Kernel 6.17)
LLM Backend	llama.cpp với lượng tử hóa GGUF Q4_K_M

4. Kế hoạch Triển khai và Đánh giá

4.1 Các Giai đoạn Phát triển Chính

Quá trình triển khai hệ thống tập trung trước tiên vào việc Thu nhận Dữ liệu (Data Ingestion) và Động cơ Quy tắc Tier 1 (Tier 1 Rule Engine). Hai bộ dữ liệu chuẩn được lựa chọn để đánh giá là:

- **CSE-CIC-IDS2018 [8]:** Tiêu chuẩn thực tế cho nghiên cứu phát hiện xâm nhập mạng doanh nghiệp. Bộ dữ liệu cung cấp cả tệp PCAP thô và các đặc trưng luồng được trích xuất sẵn, giúp so sánh trực tiếp với các nghiên cứu trước đó.
- **DAPT2020 [9]:** Bộ dữ liệu được thiết kế riêng cho việc phát hiện tấn công APT đa giai đoạn. Bộ dữ liệu này bổ sung cho CSE-CIC-IDS2018 bằng cách cung cấp các chuỗi tấn công dài hạn nhằm thử nghiệm tính năng Tác tử Tier 2 và Bộ nhớ Rủi ro Dài hạn của SENTINEL.

Cơ chế theo dõi phiên (session-tracking) được xây dựng bằng cách sử dụng Redis TTL để thiết lập đường cơ sở hành vi bình thường.

Tiếp theo, lớp Guardrail được triển khai, tích hợp thuật toán Drain3 cho gom nhóm log và phát triển mô-đun Đóng gói Dữ liệu có Phân cách bằng mật mã để trung hòa các payload độc hại.

Bước quan trọng kế tiếp là xây dựng Tác tử Tier 2 bằng LangGraph và cơ sở tri thức Dual-RAG. Hai chỉ mục FAISS riêng biệt được tạo lập: một chứa các kỹ thuật MITRE ATT&CK và một chứa quy trình ứng phó sự cố NIST SP 800-61r2.

Giai đoạn cuối cùng là việc tích hợp Bảng điều khiển Streamlit và Bộ nhớ Dài hạn bằng SQLite. Bước này hoàn thiện chu trình khép kín của hệ thống bằng cách kích hoạt tính năng Human-in-the-Loop, cho phép quản trị viên phê duyệt các bộ quy tắc tự động do LLM sinh ra và đẩy ngược chúng trở lại Tier 1. Ngoài ra, giao diện tích hợp mô-đun Điều tra Đối tượng Tương tác (Interactive Threat Investigation) cho phép nhấp chọn trực tiếp địa chỉ IP trên các bảng giám sát để hiển thị hồ sơ rủi ro chi tiết (reputation score, incidents, blocks, alerts) và lịch sử cảnh báo theo thời gian thực từ SQLite.

4.2 Kế hoạch Đánh giá

Nhằm kiểm chứng tính hiệu quả của hệ thống, nghiên cứu áp dụng chiến lược đánh giá toàn diện dựa trên Khung Đánh giá 5 Chiều (5-Dimensional Evaluation Framework).

Để đánh giá mức cơ sở, một Thí nghiệm Loại bỏ (Ablation Study) sẽ được tiến hành. Quá trình này so sánh hệ thống Hai tầng hoàn chỉnh với một kiến trúc chỉ sử dụng LLM đơn thuần để quan sát sự khác biệt về hiệu năng. Các tiêu chí đánh giá bao gồm:

- **1. Phân loại (Classification):** Điểm F1-Score, Precision, Recall và Tỷ lệ Dương tính giả (FPR), được phân tích ý nghĩa thống kê bằng kiểm định McNemar ($p < 0.05$).
- **2. Vận hành (Operational):** Độ trễ Xử lý (đại diện cho Thời gian Phát hiện/Phản hồi Trung bình), Tỷ lệ Leo thang HITL (Escalation Rate), và Tỷ lệ Cache Hit Ngữ nghĩa RAG, được kiểm định thông qua phép thử Mann-Whitney U ($p < 0.05$).
- **3. Kháng lỗi (Robustness):** Tỷ lệ Đánh bại Guardrail sẽ được kiểm thử bằng 45 mẫu adversarial đặc chế, bao trùm tấn công cấu trúc (Delimiter Smuggling) và nhầm lẫn ngữ nghĩa (Semantic Confusion).
- **4. Chất lượng Ngữ cảnh (Context Quality):** Để đánh giá suy luận của LLM và giảm thiểu Thiên kiến Tự đề cao, phương pháp LLM-as-a-Judge chéo họ được áp dụng: trong khi Gemma-2-9B-IT đóng vai trò mô hình suy luận chính, **Llama-3.1-8B-Instruct** (thuộc họ mô hình khác) sẽ làm giám khảo, chấm điểm Độ chính xác Ngữ cảnh, Sự liên quan của Câu trả lời, Tính trung thực và Độ bao phủ Ngữ cảnh theo khung RAGAS [13]. Điểm giám khảo sẽ được kiểm chứng chéo với các chỉ số RAGAS tự động để đảm bảo tính nhất quán thống kê.
- **5. Tính Giải thích (Explainability):** Tỷ lệ Hoàn thiện Nhật ký Kiểm toán (Audit Trail Completeness Rate) được đo lường một cách xác định (deterministic) thông qua kiểm tra tự động sự tồn tại của các trường dữ liệu.

4.3 Kế hoạch Thực hiện

Dự án được triển khai trong khuôn khổ 14 tuần tiêu chuẩn của Đồ án Tốt nghiệp (Capstone), cấu trúc như sau:

Bảng 2: Kế hoạch Thực hiện 14 Tuần

Tuần	Cột mốc
Tuần 1–2	Tổng quan tài liệu, thu thập dữ liệu (CSE-CIC-IDS2018) và thiết lập môi trường.
Tuần 3–4	Xây dựng Động cơ Quy tắc Tier 1 và theo dõi phiên Redis.
Tuần 5–6	Cài đặt Lớp Guardrail (Drain3 và Đóng gói có Phân cách).
Tuần 7–8	Tích hợp backend LLM cục bộ (llama.cpp) và luồng LangGraph.
Tuần 9–10	Xây dựng chỉ mục Dual-RAG FAISS (MITRE ATT&CK và NIST SP 800-61r2).
Tuần 11–12	Phát triển Bảng điều khiển Streamlit HITL và CSDL Kiểm toán SQLite.
Tuần 13	Tích hợp toàn hệ thống, chạy Thí nghiệm Loại bỏ và thu thập số liệu.
Tuần 14	Viết báo cáo tổng kết, chuẩn bị bảo vệ và nộp dự án.

5. Kết quả Kỳ vọng và Đóng góp Khoa học

5.1 Kết quả Kỳ vọng và Thực nghiệm

Nghiên cứu này đánh giá đường ống tự trị được phát triển bằng Khung đánh giá 5D. Các mục tiêu định lượng ban đầu cùng các kết quả thực tế thu thập được thông qua thử nghiệm kiểm định hệ thống được tổng hợp dưới đây:

Bảng 3: Chỉ tiêu hiệu năng định lượng kỳ vọng so với Kết quả thực nghiệm đạt được

Chiều đánh giá	Mục tiêu	Kết quả Đạt được	Cơ sở / Ghi chú
F1-Score (Phân loại)	≥ 0.90	0.982	Đánh giá trên 750 mẫu ground-truth
Giảm Độ trễ so với LLM-only	$\geq 60\%$	99.8%	Lọc nhiều hiệu quả tại Tier 1 ở tốc độ
Tỷ lệ Đánh bại Guardrail	$< 10\%$	10.0%	Đo trên 45 mẫu nghịch đảo (loại trừ)
Độ Phù hợp Ngữ cảnh (RAGAS)	≥ 0.85	0.870	Ngữ cảnh lai tích hợp MITRE ATT&
Hoàn thiện Nhật ký Kiểm toán	100%	100.0%	Kiểm tra tự động các trường bắt buộc

Việc áp dụng cơ chế Đóng gói Dữ liệu có Phân cách bằng mật mã đạt Tỷ lệ Đánh bại Guardrail là 10.0% trước các đòn tấn công prompt injection dạng cấu trúc và mã hóa (được chứng minh qua bộ kiểm thử độ bền bỉ), thiết lập một ranh giới phòng thủ vững chắc mà không làm mất đi thông tin payload quan trọng cần thiết cho việc điều tra sự cố của LLM.

Bên cạnh đó, quá trình tích hợp Dual-RAG mang lại điểm Độ Phù hợp Ngữ cảnh đạt 0.87, đảm bảo rằng Agent luôn đưa ra các đề xuất khắc phục bám sát tiêu chuẩn NIST SP 800-61r2. Về mặt vận hành, chiến lược lọc hai tầng giúp giảm trung bình 99.8% thời gian xử lý log bằng cách loại bỏ nhiều lệnh tính ở Tier 1, chứng minh tính hiệu quả của đề tài trong việc giải quyết vấn nạn alert fatigue trong các luồng SIEM lớn.

5.2 Đóng góp Khoa học

Xét về mặt học thuật, đóng góp tiên quyết của nghiên cứu là việc đề xuất một Kiến trúc Nhận thức Hai tầng độc đáo, nối liền một cách an toàn giữa sức mạnh của Động cơ Quy tắc xác định với trí tuệ của LLM Agent. Hướng tiếp cận này vẫn còn rất mới mẻ trong các văn bản nghiên cứu an ninh mạng, đặc biệt là khi kết hợp việc triển khai các LLM bảo mật cục bộ cùng với rào chắn adversarial linh hoạt.

Đóng góp thứ hai nằm ở việc xây dựng phương pháp luận đánh giá các hệ thống SOC Agent dựa trên LLM. Thông qua Khung Đánh giá 5 Chiều (5D) — kết hợp giữa phân loại thống kê, chỉ số vận hành, kháng lỗi adversarial, chấm điểm LLM-as-a-Judge chéo họ và tính giải thích — nghiên cứu cung cấp một chuẩn mực (baseline) toàn diện để các nghiên

cứu tương lai về an ninh mạng AI có thể tham chiếu.

Cuối cùng, ở góc độ thực tiễn, nghiên cứu đóng góp một pipeline mã nguồn mở chuẩn hóa (SENTINEL), tích hợp toàn vẹn từ khâu phân loại cảnh báo đến sự phản hồi có giám sát của con người. Đây là một bộ khung có tính mở rộng cao, sẵn sàng để các tổ chức ứng dụng vào môi trường thực tế nhằm giảm thiểu vấn nạn quá tải cảnh báo một cách an toàn và bền vững.

6. Kết luận

Trong nghiên cứu này, chúng tôi đề xuất một Kiến trúc Nhận thức Hai tầng kết hợp một Động cơ Quy tắc xác định tốc độ cao với một Mô hình Ngôn ngữ Lớn Tác tử có khả năng suy luận sâu. Kiến trúc này được thiết kế để giải quyết "Nghịch lý Dữ liệu lớn" và hiện tượng quá tải cảnh báo đang bào mòn năng lực của các Trung tâm Điều hành An ninh hiện đại, đồng thời giảm thiểu các lỗ hổng rủi ro đặc thù khi ứng dụng trực tiếp LLM vào luồng dữ liệu adversarial log.

Hệ thống tận dụng tốc độ của Tier 1 để sàng lọc khối lượng lớn nhiễu, và tin cậy vào các rào chắn Guardrail vững chắc — cụ thể là cơ chế Đóng gói Dữ liệu có Phân cách — để trung hòa các đòn prompt injection trước khi chúng chạm đến Agent ở Tier 2. Được trang bị bộ nhớ tri thức Dual-RAG gồm MITRE ATT&CK cho nhận diện chiến thuật và NIST SP 800-61r2 cho hướng dẫn ứng phó sự cố, Agent có khả năng đưa ra các phân tích rủi ro giàu tính ngữ cảnh và có độ chính xác cao, kèm theo các bước khắc phục có thể hành động. Hơn thế nữa, thiết kế chu trình khép kín đảm bảo sự an toàn thông qua hệ thống cách ly Human-in-the-Loop.

Những đóng góp kỳ vọng của nghiên cứu bao gồm việc đề xuất kiến trúc hybrid mới lạ, cung cấp các phương pháp đánh giá thực nghiệm cho AI Security Agent, và phát triển một pipeline thực tiễn, mở rộng được cho quy trình phản ứng sự cố tự động theo thời gian thực. Các hướng nghiên cứu trong tương lai có thể mở rộng pipeline Dual-RAG với Sigma Rules cho việc tổng hợp quy tắc tự động và MITRE D3FEND cho việc sinh phản ứng nhận biết biện pháp đối phó.

Tài liệu

- [1] Ponemon Institute, “The State of Security Operations: Alert Fatigue and Analyst Burnout,” *Annual Cybersecurity Report*, 2023.
- [2] K. Greshake, S. Abdelnabi, S. Mishra, A. Endres, T. Holz, and M. Fritz, “More than you’ve asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models,” *arXiv preprint arXiv:2302.12173*, 2023.
- [3] M. Habibzadeh, et al., “Large Language Models for Security Operations Centers: A Comprehensive Survey,” *arXiv preprint*, 2025.
- [4] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “Survey of intrusion detection systems: techniques, datasets and challenges,” *Cybersecurity*, vol. 2, no. 1, p. 20, 2019.
- [5] Z. Li, J. Ning, and B. Wang, “AttacKG: Constructing Technique Knowledge Graph from Cyber Threat Intelligence Reports,” *Findings of EACL 2023*, arXiv:2111.07093.
- [6] H. Guo, S. Yuan, and X. Wu, “LogBERT: Log Anomaly Detection via BERT,” in *2021 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2021.
- [7] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer Security Incident Handling Guide,” *NIST Special Publication 800-61 Revision 2*, 2012.
- [8] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.
- [9] S. Myneni, A. Chowdhary, A. Sabur, D. Dasgupta, et al., “DAPT 2020 — Constructing a Benchmark Dataset for Advanced Persistent Threats,” in *Workshop on Deployable Machine Learning for Security Defense (MLHat), KDD 2020*, 2020.
- [10] Y. Zhang, et al., “Agent Security Bench (ASB): Evaluating the Robustness of LLM Agents,” *arXiv preprint arXiv:2410.02644*, 2024.
- [11] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, “Drain: An Online Log Parsing Approach with Fixed Depth Tree,” in *2017 IEEE International Conference on Web Services (ICWS)*, 2017.
- [12] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” *OWASP*, 2025.

- [13] S. Shahul Es, et al., “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [14] L. Zheng, W.-L. Chiang, et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [15] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.